Intraoperation Parallelism

1 Overview

Intraoperation parallelism refers to parallelizing the execution of a single relational operation (like sort, join, or aggregation) by executing it simultaneously on different subsets of the data.

Data Parallelism

Execution of the same operation in parallel on different sets of data is called **data parallelism**. This is natural in database systems because relations contain large sets of tuples that can be partitioned across multiple processors.

Data Parallelism in Action

If a table has 10 million rows and we have 10 processors, we can assign 1 million rows/-tuples to each processor. If each processor can process 1 million rows in 1 second, the entire operation takes 1 second instead of 10 seconds.

2 Parallel Sort

Sorting can be parallelized in two primary ways depending on how the data is partitioned.

2.1 Range-Partitioning Sort

This method works in two distinct steps:

1. **Redistribution:** Tuples are redistributed using a range-partition strategy. All tuples in the i-th range are sent to processor P_i.
2. **Local Sorting:** Each processor P_i sorts its partition locally and independently.

Range-Partitioning Sort Example

Suppose we sort a relation on the `Age` attribute across 3 processors:

- P_0 receives tuples where $Age < 25$.
- P_1 receives tuples where $25 \leq Age < 50$.
- P_2 receives tuples where $Age \geq 50$.

Each processor sorts its assigned chunk. The final sorted relation is just P_0 followed by P_1 and then P_2 (simple concatenation).

2.2 Parallel External Sort–Merge

This is an alternative when data is distributed (using any method) across n disks:

1. Each processor P_i locally sorts the data on its disk D_i.
2. **Parallel Merging:**
   - Sorted partitions are range-partitioned across processors $P_0, \ldots, P_{m-1}$.
   - Each processor performs a merge on the received sorted streams.
   - The results are concatenated.

Parallel External Sort–Merge Example

Imagine sorting names across 2 processors (P_0, P_1):

1. **Local Sort:** P_0 sorts its data to get `[Alice, Zeke]`. P_1 sorts its data to get `[Bob, Yoyo]`.
2. **Redistribution (Range Partitioning):** Range A-M is assigned to P_0, N-Z to P_1.
   - P_0 sends `Zeke` to P_1.
   - P_1 sends `Bob` to P_0.
3. **Merge:**
   - P_0 merges `Alice` and `Bob` $\rightarrow$ `[Alice, Bob]`.
   - P_1 merges `Yoyo` and `Zeke` $\rightarrow$ `[Yoyo, Zeke]`.

Result: P_0 followed by P_1 gives `[Alice, Bob, Yoyo, Zeke]`.

Communication Skew in Merging

If every processor sends partition 0 to P_0 first, then partition 1 to P_1, receiving becomes sequential. **Solution:** Each processor should cycle through partitions, sending one block to each in every round to ensure all receivers work in parallel.

3 Parallel Join

Join parallelism attempts to split pairs of tuples to be tested over several processors.

3.1 Partitioned Join

Applicable for **equi-joins** and **natural joins**.

- Both relations r and s are partitioned into n partitions using the **same partitioning function** (hash or range) on the join attributes.
- Partition r_i and s_i are sent to processor P_i, which computes the join locally.

Partitioned Join Scenario

Joining `Employee(ID, Name, DeptID)` and `Department(DeptID, DName)`:

1. Use `hash(DeptID) % N` to partition both tables across N processors.
2. Processor P_i gets all employees and departments belonging to hash bucket i.
3. Since matching `DeptID` values always hash to the same bucket, P_i has all potential matches for its subset.

3.2 Fragment-and-Replicate Join

Partitioning is not applicable for some join conditions, such as non-equijoins (e.g., $r.A > s.B$) or inequality predicates. In such cases, the system uses the **fragment-and-replicate** technique to ensure every tuple in r can be tested against every tuple in s.

- **Asymmetric Case:** One relation (e.g., r) is partitioned using any technique (round-robin, hash, etc.), and the other relation (s) is **replicated** across all processors. Each processor P_i computes the join of partition r_i with the complete set s.
- **General Case:** Relation r is partitioned into n parts ($r_0, \ldots, r_{n-1}$) and s into m parts ($s_0, \ldots, s_{m-1}$). This requires $n \times m$ processors ($P_{0,0}$ to $P_{n-1,m-1}$). Each processor $P_{i,j}$ computes the join of r_i with s_j.

Asymmetric Fragment-and-Replicate

Scenario: Joining a massive 500GB `Transactions` table with a small 10MB `StoreBranch` table.

1. `Transactions` is already partitioned across 50 processors ($r_0, \ldots, r_{49}$).
2. Replicate the entire 10MB `StoreBranch` (s) to every one of the 50 processors.
3. **Result:** Each processor joins its 10GB of transactions with the local copy of branch data. This is cheaper than repartitioning both on join attributes.

General Fragment-and-Replicate

Scenario: Joining r (4 partitions) and s (2 partitions) across $4 \times 2 = 8$ processors.

- **Replication of r:** Fragment r_i is replicated across processors $P_{i,0}$ and $P_{i,1}$.
- **Replication of s:** Fragment s_j is replicated across processors $P_{0,j}, P_{1,j}, P_{2,j}, P_{3,j}$.
- **Computation:** Processor $P_{2,1}$ specifically joins local fragments r_2 and s_1.

This reduces the memory footprint for s at each processor compared to the asymmetric case.

Cost Considerations

Fragment-and-replicate usually has a **higher cost** than partitioned join because it involves extensive data replication. However, it is the only viable option for arbitrary join conditions and is highly efficient when one relation is small enough to fit in memory.

3.3 Partitioned Parallel Hash Join

This algorithm parallelizes the standard hash-join by distributing the workload across n processors. Assume s is the smaller relation (build relation) and r is the larger relation (probe relation).

1. **Partitioning Phase (s):** A hash function h_1 maps each tuple in s to one of the n processors. Each processor P_i reads local tuples of s from disk D_i and sends them to the destination processor determined by $h_1(join\_attr)$.
2. **Local Partitioning (s_i):** As processor P_i receives tuples (s_i), it partitions them further using a **second hash function** h_2. This h_2 is used to build the local hash table in memory.
3. **Redistribution Phase (r):** Once s is distributed, the larger relation r is redistributed using the same hash function h_1.
4. **Build and Probe:** As P_i receives tuples of r (r_i), it repartitions them using h_2 to probe the local hash table and produce results.

Parallel Hash Join Scenario

Joining `Orders` (r) and `Customers` (s) on `CustomerID`:

- h_1: `CustomerID % 4` determines which of the 4 processors handles the tuple.
- h_2: A more complex hash like `MurmurHash(CustomerID)` used locally by each processor to manage its in-memory hash bucket chain.
- **Optimization:** Hybrid hash-join can be used to keep the first partition of s_i in memory to avoid disk I/O for that chunk.

3.4 Parallel Nested-Loop Join

This technique is particularly useful when one relation is small and the other is already partitioned with an index.

- **Condition:** Relation s is small, r is large and partitioned across disks. There is an **index** on the join attribute of r at each partition.
- **Mechanism:** Use **asymmetric fragment-and-replicate**. Relation s is replicated across all processors P_i that store a partition of r.
- **Execution:** Each processor P_i performs an **indexed nested-loop join** between the local partition r_i and the replicated relation s.

Indexed Parallel Nested-Loop Example

Query: Find all `VIP_Customers` (s, 1000 rows) and their `Transactions` (r, 1 Billion rows).

1. `Transactions` table is already partitioned across 100 nodes, each with a B+ Tree index on `CustomerID`.
2. Broadcast the 1000 VIP records to all 100 nodes.
3. Each node iterates through the 1000 VIPs and performs a high-speed index lookup on its local 10-million-row `Transactions` fragment.

4 Other Relational Operations

4.1 Selection $\sigma_\theta(r)$

The parallelization of selection depends on the selection condition θ:

- **Point Selection ($\theta : a_i = v$):** If r is partitioned on attribute a_i, the selection is performed at a **single processor**.
- **Range Selection ($\theta : l \leq a_i \leq u$):** If r is range-partitioned on a_i, the selection is performed only at processors whose partitions **overlap** with the specified range $[l, u]$.
- **Other Cases:** The selection must be performed in parallel across **all processors**.

4.2 Duplicate Elimination

Can be performed using two primary techniques:

- **Sorting approach:** Use parallel sort and eliminate duplicates as soon as they are found during the sorting process.
- **Partitioning approach:** Partition the tuples using range or hash partitioning on all attributes, then perform duplicate elimination **locally** at each processor.

4.3 Projection

- **Without Duplicate Elimination:** Performed in parallel as tuples are read from disk.
- **With Duplicate Elimination:** Requires one of the techniques mentioned above (sorting or partitioning).

4.4 Grouping and Aggregation

To parallelize aggregation, the relation is partitioned on the grouping attributes, and aggregate values are computed locally.

Optimized Two-Stage Aggregation

To reduce communication costs, we can perform **partial aggregation** before partitioning:

1. **Local Stage:** Each processor P_i computes partial aggregates (e.g., partial sums/-counts) for its local data on disk D_i.
2. **Redistribution:** Only the partial results are partitioned on the grouping attributes.
3. **Final Stage:** Destination processors compute the final global aggregate from the partial ones.

Benefit: Significantly fewer tuples are sent across the network compared to redistributing the raw data.

5 Cost of Parallel Evaluation

If there is no skew and no parallel overhead, the expected speed-up is $1/n$ for n processors. However, real-world performance is estimated as:

$$T_{total} = T_{part} + T_{asm} + \max(T_0, T_1, \ldots, T_{n-1})$$

Cost Components

- T_{part}: Time for partitioning the relations among processors.
- T_{asm}: Time for assembling the results from all processors.
- T_i: Time taken for the operation at processor P_i.

Performance Factors

The estimation of T_i must account for:

- **Skew:** Uneven distribution of work (the slowest processor P_{max} dictates T_{total}).
- **Contention:** Conflict for shared resources (memory, disk, network bandwidth).
- **Start-up Costs:** Overhead of initiating the operation on multiple nodes.

Intraoperation parallelism provides the foundation for massive scalability in modern database systems.